



Συστήματα Ανάκτησης Πληροφοριών

Εργασία Φάση 2 2024-2025

Κεχριώτη Ελένη

3210078

Ισουφάι Ντέιβιντ

3210056

Εισαγωγή

Στην παρούσα εργασία υλοποιείται και αξιολογείται ένα σύστημα Ανάκτησης Πληροφορίας, με στόχο την αποτελεσματική αναζήτηση επιστημονικών εγγράφων. Το σύστημα βασίζεται στο **Elasticsearch**, μια ισχυρή μηχανή αναζήτησης ανοικτού κώδικα, και εφαρμόζεται πάνω στο σύνολο δεδομένων **SciDocs**, το οποίο περιλαμβάνει μεταδεδομένα, τίτλους και κείμενα επιστημονικών άρθρων. Η επικοινωνία με την Elastic Search γίνεται μέσω python client και της βιβλιοθήκης elasticsearch.

Στην παρούσα φάση επεκτείνουμε το σύστημα IR που αναπτύξαμε στη Φάση 1, ενσωματώνοντας σημασιολογική πληροφορία μέσω της χρήσης συνωνύμων από το WordNet. Στόχος είναι η βελτίωση της ανάκτησης σχετικών εγγράφων, μέσω εμπλουτισμού των queries με λέξεις παρόμοιας σημασίας. Για την επέκταση αυτή, αξιοποιήθηκαν δύο διαφορετικές μεθοδολογίες: η χρήση του synonym_graph filter της Elasticsearch και η χρήση της βιβλιοθήκης NLTK σε Python.

Για την εργασία θα πρέπει να κατεβάσετε τις βιβλιοθήκες elasticsearch και nltk.

```
> pip install nltk elasticsearch
```

Μεθοδολογία

Η μεθοδολογία της εργασίας ακολουθεί τα παρακάτω βασικά βήματα:

1. Σενάρια

Σενάριο	Μέθοδος	Περιγραφή
A	Elasticsearch	Χρήση αρχείου wn_s_nouns_verbs.pl σε synonym_graph filter ώστε τα queries να επεκτείνονται αυτόματα κατά την αναζήτηση.
B	NLTK	Επέκταση των queries με υπερώνυμα και υπώνυμα πριν την αναζήτηση.
Γ	NLTK	Επέκταση των queries με τα 2 πρώτα συνώνυμα ανά λέξη, βάσει POS tagging με NLTK.

2. Λεπτομέρειες υλοποίησης

- Elasticsearch σενάριο (A)
 - Δημιουργήθηκε custom analyzer με το synonym_graph filter.

```
"analyzer": {  
  "default": {  
    "type": "english"  
  },  
  "default_search": {  
    "tokenizer": "standard",  
    "filter": [  
      "lowercase",  
      "english_stemmer",  
      "wn_synonym_filter",  
      "english_stop"  
    ]  
  }  
}
```

- Τα αρχεία wn_s_nouns_verbs.pl/wn_s.pl/wn_s_nouns.pl προστέθηκαν ως λίστα συνωνύμων.

```
"wn_synonym_filter": {  
  "type": "synonym_graph",  
  "format" : "wordnet",  
  "synonyms_path": "analysis/wn_s.pl"  
},
```

- Ο αναλυτής εφαρμόζεται κατά την εκτέλεση των queries, οπότε η αναζήτηση μας παραμένει όπως και στην Φάση 1.

- NLTK σενάριο (B)
 - Πραγματοποιήθηκε επεξεργασία των queries σε Python με NLTK και WordNet.
 - Για κάθε όρο του query, εξήχθησαν υπερώνυμα (hypernyms) και υπόωνυμα (hyponyms), ανάλογα με το μέρος του λόγου.

```
def get_hypernyms(word):
```

```

hypernyms = set()

for syn in wn.synsets(word):
    for hypernym in syn.hypernyms():
        for lemma in hypernym.lemmas():
            hypernyms.add(lemma.name().replace('_', ' '))

return hypernyms

def get_hyponyms(word):
    hyponyms = set()

    for syn in wn.synsets(word):
        for hyponym in syn.hyponyms():
            for lemma in hyponym.lemmas():
                hyponyms.add(lemma.name().replace('_', ' '))

    return hyponyms

def wordnet_features(text):
    words = text.split()

    features = []

    for word in words:
        hyponyms = get_hyponyms(word)
        hypernyms = get_hypernyms(word)

        features.extend(hyponyms)
        features.extend(hypernyms)

    return ' '.join(features)

```

- Τα επαυξημένα queries δημιουργήθηκαν εξωτερικά και στάλθηκαν απευθείας στην Elasticsearch για αναζήτηση.

```

query_obj = json.loads(line)
query_id = query_obj["_id"]
query_text = query_obj["text"]

expanded_query_str = wordnet_features(query_text)

```

```

response = client.search(
    index=index_name,
    size=topk,
    query={
        "bool": {
            "should": [
                {"match": { "title": {"query":
expanded_query_str,"boost": 1}}},
                {"match": { "text": {"query":
expanded_query_str,"boost": 1}}},
                {"match": { "allContent": {"query":
expanded_query_str,"boost": 1}}}
            ]
        }
    }
)

```

- NLTK σενάριο (Γ)
 - ο Εκτελέστηκε POS tagging με NLTK για όλους τους όρους των queries.

```

def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wn.ADJ
    elif treebank_tag.startswith('V'):
        return wn.VERB
    elif treebank_tag.startswith('N'):
        return wn.NOUN
    elif treebank_tag.startswith('R'):
        return wn.ADV
    else:
        return None

```

- ο Για κάθε όρο, επιλέχθηκαν τα 2 πρώτα συνώνυμα από τα αντίστοιχα synsets στο WordNet.

```

def expand_query_wordnet_smart(query):
    stop_words = set(stopwords.words('english')) +
list(string.punctuation))
    tokens = word_tokenize(query.lower())
    tagged = pos_tag(tokens)

    expanded = set()
    for word, tag in tagged:
        if word in stop_words:
            continue

        wn_tag = get_wordnet_pos(tag)
        if wn_tag is None:
            continue

        expanded.add(word)

    synsets = wn.synsets(word, pos=wn_tag)

```

```

if synsets:
    for lemma in synsets[0].lemmas()[2]:
        synonym = lemma.name().replace('_', ' ')
        if synonym != word:
            expanded.add(synonym)

return " ".join(expanded)

```

- Τα queries επεκτάθηκαν με τους νέους όρους και υποβλήθηκαν απευθείας στην Elasticsearch.

```

query_obj = json.loads(line)
query_id = query_obj["_id"]
query_text = query_obj["text"]

expanded_query_str = expand_query_wordnet_smart(query_text)

response = client.search(
    index=index_name,
    size=topk,
    query={
        "bool": {
            "should": [
                {"match": {"title": {"query": query_text, "boost": 3}}},
                {"match": {"text": {"query": query_text, "boost": 3}}},
                {"match": {"allContent": {"query": query_text, "boost": 3}}},
                {"match": {"title": {"query": expanded_query_str, "boost": 1}}},
                {"match": {"text": {"query": expanded_query_str, "boost": 1}}},
                {"match": {"allContent": {"query": expanded_query_str, "boost": 1}}}
            ]
        }
    }
)

```

3. Πειραματική διαδικασία

- Elasticsearch σενάριο (A)

Η πειραματική διαδικασία ξεκίνησε στο πρώτο σενάριο, όπου δοκιμάσαμε τον ίδιο analyzer με διαφορετικά αρχεία συνωνύμων για την αναζήτηση.

- Αναζήτηση με την χρήση του αρχείου **wn_s.pl** ως φίλτρου συνωνύμων στο ευρετήριο (για απλότητα παραθέτουμε μόνο τις μετρικές για τα πρώτα 5 και 10 ανακτηθέντα κείμενα και μόνο για 20 queries).

Trec Eval Evaluation for file my_results20.run

recall_5	0.0689
recall_10	0.0989
map	0.0577
P_5	0.0677
P_10	0.0487

- Αναζήτηση με την χρήση του αρχείου **wn_s_nouns.pl** ως φίλτρου συνωνύμων στο ευρετήριο (για απλότητα παραθέτουμε μόνο τις μετρικλες για τα πρώτα 5 και 10 ανακτηθέντα κείμενα και μόνο για 20 queries).

Trec Eval Evaluation for file my_results20.run

recall_5	0.0847
recall_10	0.1245
map	0.0720
P_5	0.0835
P_10	0.0615

- Αναζήτηση με την χρήση του αρχείου **wn_s_nouns_verbs.pl** ως φίλτρου συνωνύμων στο ευρετήριο (για απλότητα παραθέτουμε μόνο τις μετρικλες για τα πρώτα 5 και 10 ανακτηθέντα κείμενα και μόνο για 20 queries).

Trec Eval Evaluation for file my_results20.run

recall_5	0.0726
recall_10	0.1062
map	0.0605
P_5	0.0713
P_10	0.0523

- NLTK σενάριο (B)

- Η πειραματική διαδικασία συνεχίστηκε με το σενάριο B και την χρήση υπερώνυμων (hypernyms) και υπόωνυμων (hyponyms). Ο συνδυασμός αυτός επιλέχθηκε μεταξύ άλλων έπειτα από δοκιμές ως ο καλύτερος συνδυασμός. Παρόλα αυτά όμως βλέπουμε πως οι μετρικές μας σε σχέση με το Σενάριο A μειώθηκαν.

Trec Eval Evaluation for file my_results20.run

recall_5	0.0690
recall_10	0.0951
map	0.0556
P_5	0.0682
P_10	0.0469

- NLTK σενάριο (Γ)

- Τελικά καταλήξαμε στο Σενάριο Γ, όπου με τον τρόπο που λειτουργεί (όπως περιγράφηκε στην ενότητα 2) καταφέραμε να πετύχουμε καλύτερα σκορ συγκριτικά με τα άλλα δύο σενάρια. Όπως ίσως παρατηρήσατε και πάνω στο σενάριο Γ στο search query χρησιμοποιείται τόσο το επεκτεταμένο query όσο και το αρχικό με το πρώτο να έχει boost 1 και το δεύτερο boost 3. Καταλήξαμε σε αυτήν την υλοποίηση διότι παρατηρήσαμε ότι από μόνο του το expanded query δεν βελτιώνει τα score, αλλά τα μειώνει. Κάτι που όμως είναι αρκετά αναμενόμενο αν αναλογιστούμε ότι τα κείμενα μας περιέχουν επιστημονικούς όρους, οι οποίοι θέλουν υψηλή ακρίβεια κατά την αναζήτηση. Η επέκταση με συνώνυμα, ακόμα και με τα δύο πρώτα συνώνυμα, η ακρίβεια αυτή χάνεται ελαφρώς. Συνεπώς, αποφασίσαμε να δώσουμε μεγαλύτερη βαρύτητα στο original query, διατηρώντας το expanded query ως συμπληρωματική ενίσχυση.

Trec Eval Evaluation for file my_results20.run

recall_5	0.1178
recall_10	0.1649
map	0.1010

P_5	0.1162
P_10	0.0814

4. Τελική υλοποίηση

Από τα πειράματα που υλοποιήθηκαν στο πλαίσιο της Φάσης 2, των οποίων τα αποτελέσματα συνοψίζονται στον παρακάτω συγκεντρωτικό πίνακα, καταλήξαμε στο σενάριο Γ για την τελική μας υλοποίηση.

Σενάριο	MAP	Precision@20	Recall@20
Φάση Α	0.1019	0.0541	0.2201
A - Elasticsearch	0.0577	0.0359	0.1457
B - NLTK υπερώνυμα	0.0556	0.0469	0.0951
Γ- NLTK (pos tagging)	0.1010	0.0542	0.2201

Σενάριο	Φάση Α	Γ- NLTK (pos tagging)
recall_5	0.1176	0.1178
recall_10	0.1675	0.1649
recall_15	0.1974	0.1969
recall_20	0.2196	0.2201
recall_30	0.2196	0.2201
recall_100	0.2196	0.2201
recall_200	0.2196	0.2201
recall_500	0.2196	0.2201
recall_1000	0.2196	0.2201

Όπως μπορούμε να παρατηρήσουμε, **το MAP** του σεναρίου Γ **μειώθηκε** σε σχέση με την Φάση Α, ενώ **το recall** για κάθε top k ανακτηθέντα κείμενα **αυξήθηκε**. Αυτό συμβαίνει διότι η χρήση συνωνύμων για την αναζήτηση, διευρύνει τους όρους της αναζήτησης, εντοπίζοντας περισσότερα σχετικά κείμενα ακόμα και αν δεν περιέχουν τον αρχικό όρο,, αλλά ταυτόχρονα εισάγει και πολύ “θόρυβο” (κείμενα που δεν είναι τόσο σχετικά με το query) μειώνοντας έτσι την ακρίβεια.

5. Αποτελέσματα

Τα αποτελέσματα για τα τοπ 20,30,50 του MAP καθώς και η μέση ακρίβεια για τα πρώτα 5,10,15,20 φαίνεται παρακάτω.

	20	30	50
map	0.1010	0.1040	0.1066
avgPre@5	0.1162	0.1162	0.1162
avgPre@10	0.0814	0.0814	0.0814
avgPre@15	0.0647	0.0647	0.0647
avgPre@20	0.0542	0.0542	0.0542